

הנדסת מערכות חלונאיות

עקרונות, תבניות וכלים מבוססי פייתון ו-C#



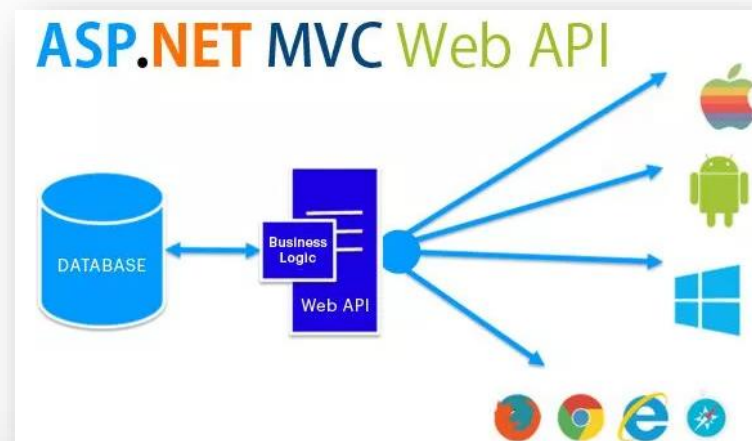
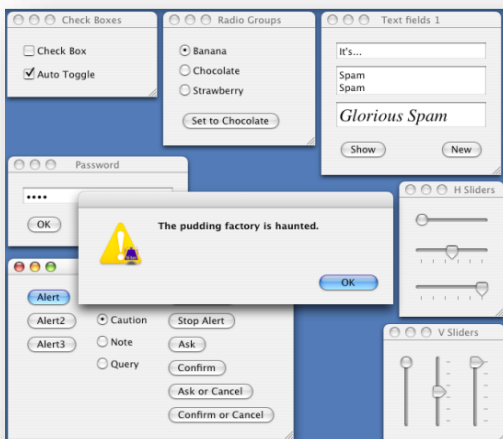
יוסי זגורי

Yossi.Zaguri@gmail.com

052-4668866

מטרות הקורס

- ✓ הכרת מושגים ותבניות עיצוב רלוונטיות להנדסת מערכות
Desktop ו- Cross Platforms Apps
- ✓ הכרת תשתיות חוצות פלטפורמה לפיתוח תצוגה בסביבת פיתון 3.x
- ✓ הטמעת גישת ותבניות UI/UX בעיצוב תצוגה
- ✓ עיצוב ופיתוח שירותים מבוססי ASP.NET Web API ואינטגרציית
שירותי ענן במערכות Desktop



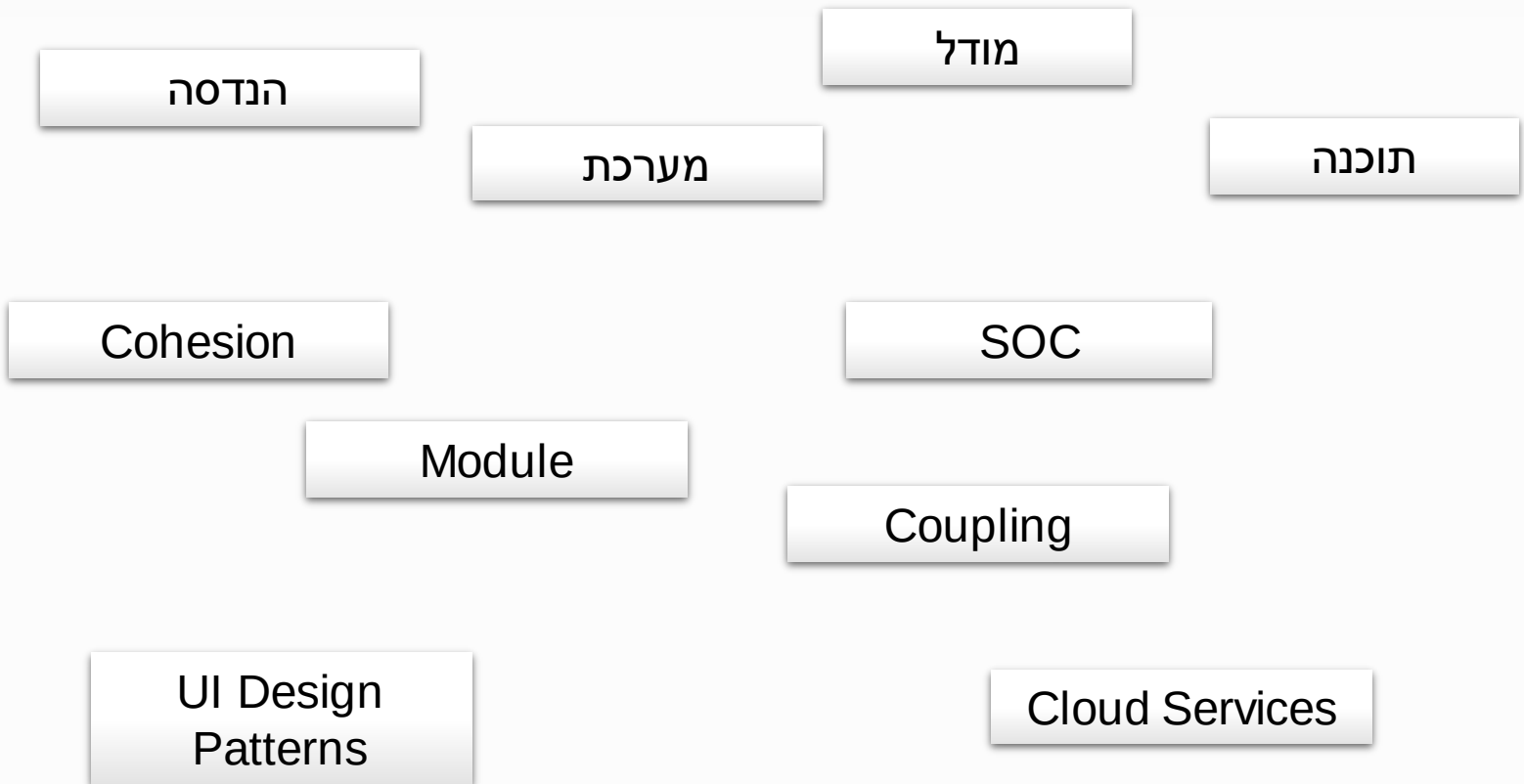


If you wish to converse with me,
define your terms.

~ Voltaire

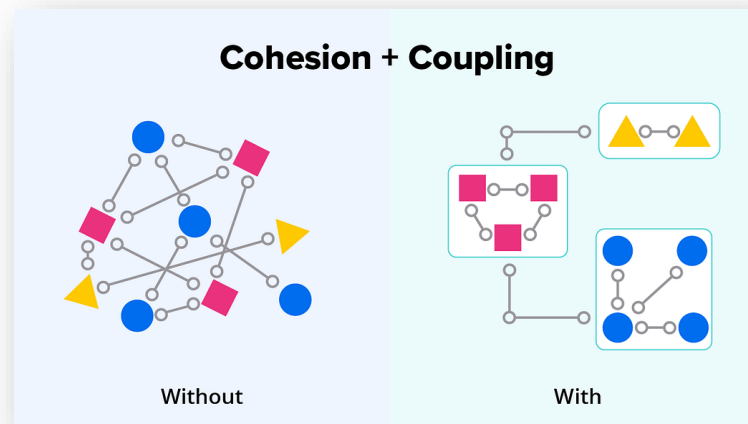
AZ QUOTES

מושגים

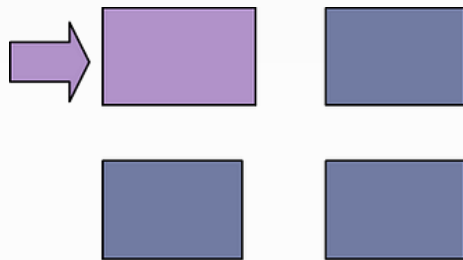


Soc & modulation

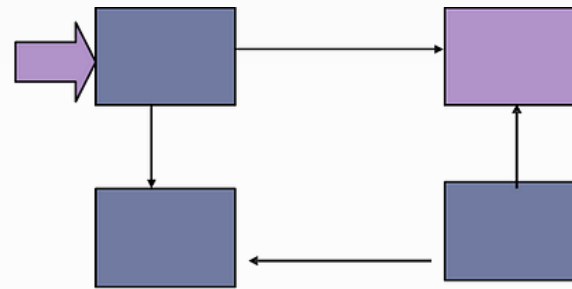
- ❑ Separation of Concerns : Fundamental principle of software development and architecture, ensuring that different concerns do not **coexist** within the design or code
- ❑ Related to SOLID'S Single Responsibility Principle (SRP)
- ❑ Separate concepts and code to **modules**
- ❑ Each module should be loosely coupled from other modules and highly cohesive within itself
- ❑ Interfaces: Major technique to achieve loose coupling (along side IoC methods)



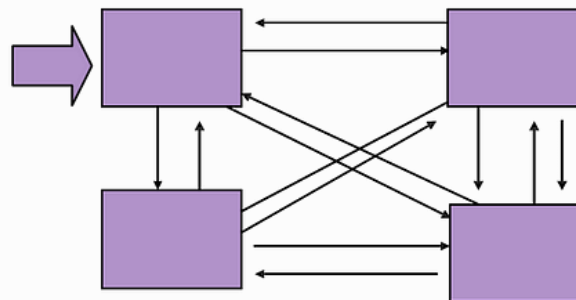
Soc & modulation



Uncoupled



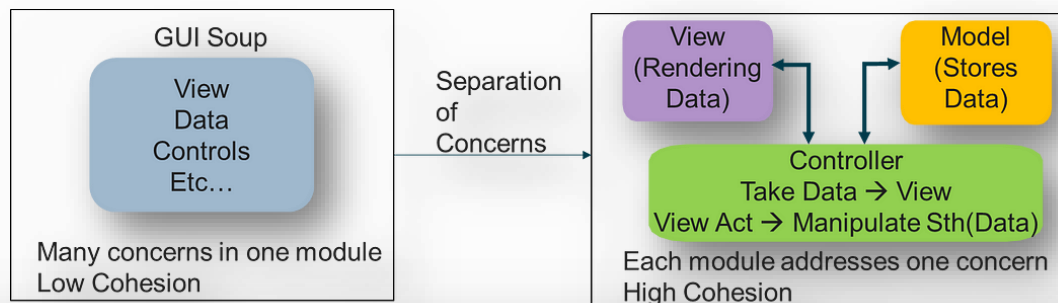
Loosely Couple: Some Dependencies



'ripple' effect

Highly Couple: Many Dependencies

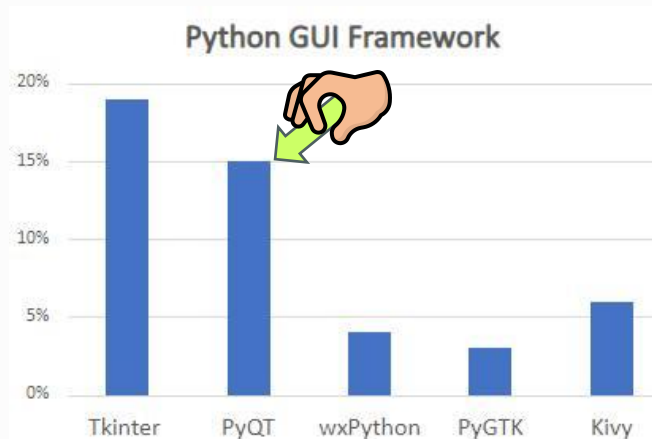
- ❑ Graphical User Interfaces (GUIs) allow users to interact with software through intuitive and user-friendly graphical elements such as buttons, icons, text boxes, and windows.
- ❑ GUIs provide a simple way to perform complex tasks and navigate a software system using the mouse and keyboard and without needing to remember complex commands.
- ❑ Using familiar visual concepts across different tools and platforms helps making new software feel intuitive and user-friendly.
- ❑ Two relevant design aspects:
 - ❑ Structural code modulation and design patterns including MV*, Commands, Strategy, Repository, DI, caching, Factories etc. [Importance of Design Patterns](#) aims to promote reusability, maintainability, testability etc.
 - ❑ UI/UX : Look & Feel, total users experience with the system, aimed to maximize usage attributes including task completion time, error rates etc., [Common UI Design Patterns](#)



GUI & Python

- ❑ Python is commonly used for command-line tools, data science, and web apps.
- ❑ It also perfectly capable of building graphical desktop applications.
- ❑ Many libraries exist (<https://www.pythonguis.com/faq/which-python-gui-library/>)
- ❑ Option 1: Tkinter bundled with python, simple and suited for small portable apps.
- ❑ Option 2: PyQt is a complete GUI application development framework for building full featured desktop apps.

<https://www.pythonguis.com/faq/pyqt-vs-tkinter/>



<https://www.pythonguis.com/faq/pyqt-vs-tkinter/>



Python Reminder

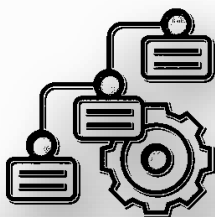
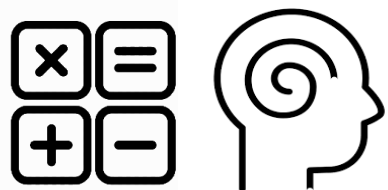
תזכורת פייתון בסיסי



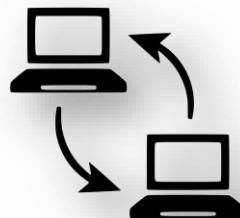
Programming Aspects



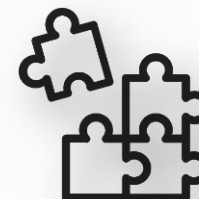
Calculations/Logic/Workflow



I/O & Communication



Split to modules



Programming Aspects



Calculations/Logic/Workflow

```
x=5+4  
  
if (x==9) :  
    print("nine")  
else:  
    print("not nine")
```

I/O & Communication

```
name=input("enter your name:")  
print("hello "+name)
```

Split to modules

```
def calc(x,y):  
    return x+y  
  
z=calc(5,4)
```

Any "part" of the code is by definition a **module**,
yet in python this term signifies a code file with *.py extension

Python – “print” output

```
a = 5
```

```
print('The value of a is', a)
```

```
print(1, 2, 3, 4, sep=',', end='!') => 1,2,3,4!
```

```
x = 5
```

```
y = 10
```

```
print('The value of x is {} and y is {}'.format(x,y)) => The value of x is 5 and y is 10
```

```
print('I love {0} and {1}'.format('Schnitzel','Chips'))=> I love Schnitzel and Chips
```

```
print(values, sep=' ', end='\n', file=sys.stdout, flush=False)
```

Python reserved keywords

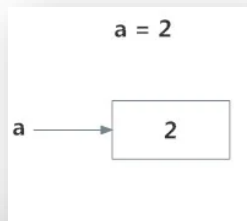


False	await	else	import	pass
None	break	except	in	raise
True	class	finally	is	return
and	continue	for	lambda	try
as	def	from	nonlocal	while
assert	del	global	not	with
async	elif	if	or	yield

Python Identifiers

- ❖ Identifier (name) is simply a name given to objects. Everything in Python is an **object**. Name is a way to access the underlying object.
- ❖ An identifier is a name given to entities like class, **functions**, **variables**, etc. It helps to differentiate one entity from another.
- ❖ Don't use keywords, start name with a letter and follow with (a to z), (A to Z), (0 to 9) or underscore(_).
- ❖ Python is a case-sensitive language.
- ❖ Give identifiers **meaningful** names.
- ❖ Id(identifier) returns address in memory

```
a = 2  
print(id(a))
```



```
FirstName="Dani"  
lastName="Levi"  
Full_Name="Dani Levi"
```

Python – Variables ,Constants & Literals

המרכז
ללימודים
במדעי המחשב



- ❖ A variable is a memory named location used to store data.
- ❖ Variables are being **Declared** and values are **Assigned** to them in several styles

```
university= "ariel.ac.il"  
x=90  
y=3.2  
a, b, c = 5, 3.2, "Hello"  
a1 = a2 = a3 = "same"
```

- ❖ Variables starting with ‘_’ or ‘__’ usually have special role in the code.
- ❖ Constants are written in all capital letters as a convention (PI = 3.14, GRAVITY = 9.8)
- ❖ Literals are explicit values embedded within the code.

```
Name="rina"  
Value=102.4  
IsOk=True  
Food=None #special value
```

```
a = 0b1010 #Binary Literals  
b = 100 #Decimal Literal  
c = 0o310 #Octal Literal  
d = 0x12c #Hexadecimal Literal
```

Python – Variable Types

- ❖ You can **get** the data type of any variable by using the `type()` function
- ❖ You can **set** the data type using the **casting functions** (if it is possible to cast)

Text Type:	str
Numeric Types:	int, float, complex
Sequence Types:	list, tuple, range
Mapping Type:	dict
Set Types:	set
Boolean Type:	bool

```
x = "5.0"  
y = float(x)  
print(type(y))
```

```
x = 5  
print(type(x))
```

```
x = str("Hello World")  
x = int(20)  
x = float(20.5)  
x = list(("apple", "banana", "cherry"))  
x = tuple(("apple", "banana", "cherry"))  
x = range(6)  
x = dict(name="John", age=36)  
x = set(("apple", "banana", "cherry"))
```

Python Collections

- ❖ **List** is defined using []. An ordered sequence of items. The items in a list do not need to be of the same type. Index starts at 0.

```
fruits = ["apple", "mango", "orange"] #list
```

- ❖ **Tuple** is an ordered sequence of items same as a list defined using (). The difference is that tuples are immutable (once created cannot be modified).

```
numbers = (1, 2, 3) #tuple
```

- ❖ **Set** is an unordered collection of unique items. Set is defined by values separated by comma inside braces { }. Items in a set are not ordered and can't have duplicates.

```
vowels = {'a', 'e', 'i', 'o', 'u'} #set
```

- ❖ Dictionary is an unordered collection of key-value pairs. use key to retrieve the respective value.

```
alphabets = {'a':'apple', 'b':'ball', 'c':'cat'} #dictionary
```

Python Collections

- ❖ **List** is defined using []. An ordered sequence of items. The items in a list do not need to be of the same type. Index starts at 0.

```
fruits = ["apple", "mango", "orange"] #list
numbers = (1, 2, 3) #tuple
alphabets = {'a':'apple', 'b':'ball', 'c':'cat'} #dictionary
vowels = {'a', 'e', 'i', 'o', 'u'} #set
```

- ❖ [] can be used to access individual items using their index, or slice a portion of the list.

```
a = [5,10,15,20,25,30,35,40]
print("a[2] = ", a[2])      #a[2] = 15
print("a[0:3] = ", a[0:3]) # a[0:3] = [5, 10, 15]
print("a[5:] = ", a[5:])    # a[5:] = [30, 35, 40]
```

Python Statements

- ❖ Instructions that a Python interpreter can execute are called **statements**
 - ❖ `a = 1` is an **assignment** statement
 - ❖ `If / for / while` are **flow-control** statements
- ❖ A **code block** (body of a function, loop, etc.) starts with indentation and ends with the first un-indented line.

```
for i in range(1,11):  
    print(i)  
    if i == 5:  
        break
```

- ❖ Comments are text remarks starting with #

```
#This is a clear comment
```

- ❖ Docstrings appear right after the definition of a function, class, or a module.
using triple quotes.

```
def multiply(num):  
    """Function to double the value"""  
    return 2*num  
print(multiply.__doc__)
```

Python Expressions

- ❖ Combination of Operands and Operator is called an ***Expression***
- ❖ Operands can be ***Literals or Variables***
- ❖ Operators – see lists
 - ❖ Expression is being ***evaluated***
 - ❖ Arithmetic Expression – result is a number
 - ❖ Logic Expression – result is True or False



Python Arithmetic Operators

<u>Operator</u>	<u>Meaning</u>	<u>Example</u>
+	Add two operands or unary plus	$x + y + 2$
-	Subtract right operand from the left or unary minus	$x - y - 2$
*	Multiply two operands	$x * y$
/	Divide left operand by the right one (always results into float)	x / y
%	Modulus - remainder of the division of left operand by the right	$x \% y$ (remainder of x/y)
//	Floor division - division that results into whole number adjusted to the left in the number line	$x // y$
**	Exponent - left operand raised to the power of right	$x ** y$ (x to the power y)

Python Logic Operatos



<u>Operator</u>	<u>Meaning</u>	<u>Example</u>
>	Greater than - True if left operand is greater than the right	$x > y$
<	Less than - True if left operand is less than the right	$x < y$
==	Equal to - True if both operands are equal	$x == y$
!=	Not equal to - True if operands are not equal	$x != y$
>=	Greater than or equal to - True if left operand is greater than or equal to the right	$x >= y$
<=	Less than or equal to - True if left operand is less than or equal to the right	$x <= y$

Python Assignments Operators

<u>Operator</u>	<u>Example</u>	<u>Equivalent to</u>
=	x = 5	x = 5
+=	x += 5	x = x + 5
-=	x -= 5	x = x - 5
*=	x *= 5	x = x * 5
/=	x /= 5	x = x / 5
%=	x %= 5	x = x % 5
//=	x //= 5	x = x // 5
**=	x **= 5	x = x ** 5
&=	x &= 5	x = x & 5
=	x = 5	x = x 5
^=	x ^= 5	x = x ^ 5
>>=	x >>= 5	x = x >> 5
<<=	x <<= 5	x = x << 5

Python Combining Conditions

<u>Operator</u>	<u>Meaning</u>	<u>Example</u>
and	True if both the operands are true	x and y
or	True if either of the operands is true	x or y
not	True if operand is false (complements the operand)	not x

Python manipulate bits



<u>Operator</u>	<u>Meaning</u>	<u>Example</u>
&	Bitwise AND	$x \& y = 0$ (0000 0000)
	Bitwise OR	$x y = 14$ (0000 1110)
~	Bitwise NOT	$\sim x = -11$ (1111 0101)
^	Bitwise XOR	$x \wedge y = 14$ (0000 1110)
>>	Bitwise right shift	$x >> 2 = 2$ (0000 0010)
<<	Bitwise left shift	$x << 2 = 40$ (0010 1000)

Python Additional Operators

Identity operators

Operator	Meaning	Example
is	True if the operands are identical (refer to the same object)	x is True
is not	True if the operands are not identical (do not refer to the same object)	x is not True

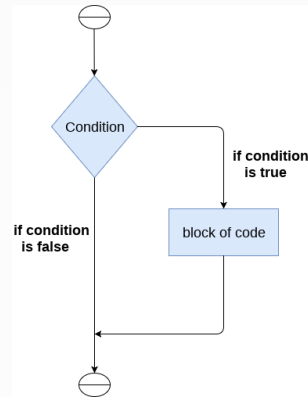
Membership operators

Operator	Meaning	Example
in	True if value/variable is found in the sequence	5 in x
not in	True if value/variable is not found in the sequence	5 not in x

Python flow control using If..else

- ❖ The if...elif...else statement is used in Python for decision making.

```
if <test expression>:  
    statement(s)  
else:  
    statement(s)
```



```
num = int(input("enter number:"))  
if num > 0:  
    print(num, "is a positive number.")  
elif num < 0:  
    print(num, "is a negative number.")  
else:  
    print(num, "equals to zero")
```

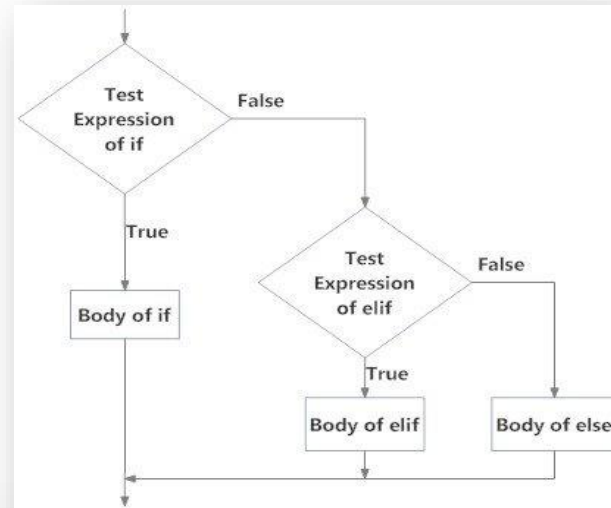


Fig: Operation of if...elif...else statement

Python flow control using for loop

- ❖ for loop in Python is used to iterate over a sequence (list, tuple, string) or others

```
for val in sequence:  
    loop body
```

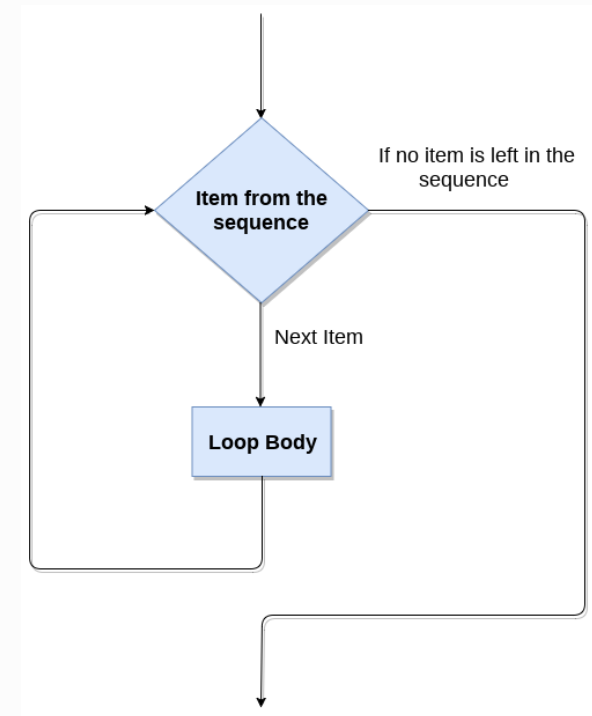
```
numbers = [6, 5, 3, 8, 4, 2, 5, 4, 11]
```

```
# variable to store the sum  
sum = 0
```

```
# iterate over the list
```

```
for val in numbers:  
    sum = sum+val
```

```
print("The sum is", sum)
```



Python Combine range() with for loop



❖ *Generate a sequence of numbers using range() function*

```
range(start, stop, step_size)
```

```
print(list(range(10))) => [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
print(list(range(2, 8))) => [2, 3, 4, 5, 6, 7]
print(list(range(2, 20, 3))) => [2, 5, 8, 11, 14, 17]
```

```
genre = ['pop', 'rock', 'jazz']

# iterate over the list using index
for i in range(len(genre)):
    print("I like", genre[i])
```

```
student_name = 'rina'
marks = {'bina': 90, 'rina': 85, 'zina': 77}

for student in marks:
    if student == student_name:
        print(marks[student])
        break
else:
    print('No entry with that name found.')
```

Python flow control using while loop

המכון
לב
האקדמיה



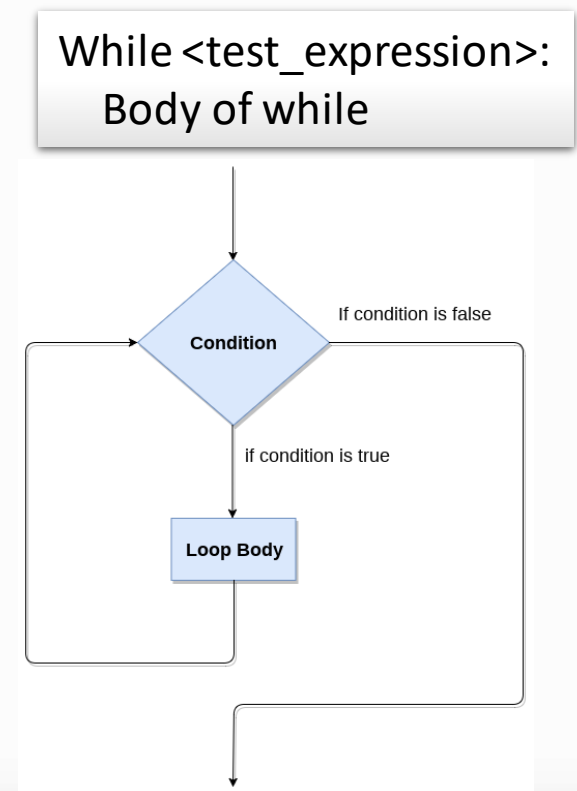
- ❖ while loop is used to iterate over a block of code as long as the test expression (condition) is true.
- ❖ The while loop can be terminated with a break statement. In such cases as with the for loop, the else part is ignored.

```
n = int(input("Enter n: "))

sum = 0
i = 1

while i <= n:
    sum = sum + i
    i = i+1  # update counter

print("The sum of 1..{0} is {1}".format(n, sum))
```





BUILT IN FUNCTIONS

Just for Fun – “Zen of Python”



In 1999, software developer Tim Peters, a major contributor to Python and creator of the original CPython implementation, wrote the “[Zen of Python](#),” an explanation of Python’s design philosophy, and the philosophy that programmers should incorporate into their programming approach. The document was later incorporated into official Python documentation.

Beautiful is better than ugly.

Explicit is better than implicit.

Simple is better than complex.

Complex is better than complicated.

Flat is better than nested.

Sparse is better than dense.

Readability counts.

Special cases aren’t special enough to break the rules.

Although practicality beats purity.

Errors should never pass silently.

Unless explicitly silenced.

In the face of ambiguity, refuse the temptation to guess.

There should be one—and preferably only one—obvious way to do it.

Although that way may not be obvious at first unless you’re Dutch.

Now is better than never.

Although never is often better than *right* now.

If the implementation is hard to explain, it’s a bad idea.

If the implementation is easy to explain, it may be a good idea.

Namespaces are one honking great idea—let’s do more of those!



Simple Types & String

```
print(type(5))  
print(type(5.0))  
print(type("5"))  
print(type(True))
```

```
x="Ariel University"  
  
print(x.upper())  
print(x.lower())  
print(x.upper())  
print(x.count("r"))  
  
y="Israel"  
print(f"We learn at {x} in {y}")  
  
z=x.replace("Ariel","Jerusalem")  
print(z)
```

Collections - List

Define and Access

```
people=["dani","dina","roni","rina"]

print(people[1])
print(people[-1])

for person in people:
    print(person)
```

Add & Combine

```
people.append("david")

people.insert(1,"dana")

morePeople=["ziv","ziva"]

allPeople=people+morePeople

print(allPeople)
```

Slicing

```
print(people[0:3])

print(people[1:])

print(people[:2])
```

Remove

```
Last=allPeople.pop(-1)

del people[0]
```

Collections - List

Sorting

```
ages=[23,66,34,50]

sorted=ages.sort()

print(sorted)

rSorted=ages.sort(reverse=True)

print(rSorted)
```

Shallow vs. Deep Copy

<code>newAges = ages</code>	<code>#shallow</code>
<code>newAges=ages[:]</code>	<code>#deep</code>
<code>newAges=ages.copy()</code>	<code>#deep</code>



Object Oriented Python

❑ An object has two characteristics:

- Attributes
- Behavior

```
class Parrot:  
    pass
```

❑ An Object is an instance of a class

- Instantiation of new object involves using Special Methods in the class, among them one called instance initializer (`__init__()`).

```
obj = Parrot()
```

❑ The `__init__()` method must have a variable representing the current object (conventionally named `self`)

❑ Class has two roles:

- Blueprint for an object
- Represents all objects of its type (i.e. class attributes)

```
class Parrot:  
    species = "bird" # class attribute  
  
    def __init__(self, name, age):  
        self.name = name  
        self.age = age
```

דוגמא – תכונות ואתחולן



```
class Parrot:
    # class attribute
    species = "bird"
    #-----
    # init instance attributes
    def __init__(self, name, age):
        self.name = name
        self.age = age

# instantiate the Parrot class
blu = Parrot("Blu", 10)
woo = Parrot("Woo", 15)

# access the class attributes
print("Blu is a {}".format(blu.__class__.species))
print("Woo is also a {}".format(woo.__class__.species))

# access the instance attributes
print("{} is {} years old".format( blu.name, blu.age))
print("{} is {} years old".format( woo.name, woo.age))
```



דוגמא – הוספת מתודות

```
class Parrot:
    # instance attributes
    def __init__(self, name, age):
        self.name = name
        self.age = age

    # instance method
    def sing(self, song):
        return "{} sings {}".format(self.name, song)

    def dance(self):
        return "{} is now dancing".format(self.name)

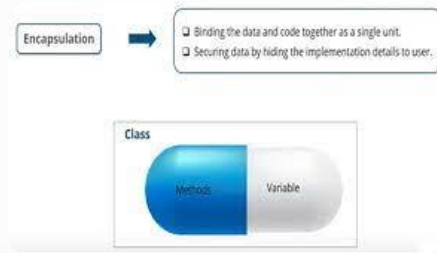
# instantiate the object
blu = Parrot("Blu", 10)

# call our instance methods
print(blu.sing("Happy"))
print(blu.dance())
```

תפיסה מונחית עצמים - כימוס

❑ In object-oriented data should be **protected** from misuse

- Called *Data Hiding*
- Classically it is being done by specifying a property is *private*, preventing direct access to it
- Python uses *conventions* : denote private attributes using underscore as the prefix, i.e single _ or double _
- Using Setter & Getter methods to access private variables
- More on double _ later...



```
class Computer:
    def __init__(self):
        self._maxprice = 900

    def sell(self):
        print("Selling Price: {}".format(self._maxprice))

    def setMaxPrice(self, price):
        self._maxprice = price

c = Computer()
c.sell()

# change the price
c._maxprice = 1000
c.sell()

# using setter function
c.setMaxPrice(1000)
c.sell()
```



דוגמא - הורשה

```
# parent class
class Bird:
    def __init__(self):
        print("Bird is ready")

    def whoisThis(self):
        print("Bird")

    def swim(self):
        print("Swim faster")

# child class
class Penguin(Bird):
    def __init__(self):
        # call super() function
        super().__init__()
        print("Penguin is ready")

    def whoisThis(self):
        print("Penguin")

    def run(self):
        print("Run faster")
```

```
peggy = Penguin()
peggy.whoisThis()
peggy.swim()
peggy.run()
```



פונקציות בנאי

- ☐ ניתן לכלול פונקציות מיוחדות במסגרת מחלקה (נקראות Magic Methods)
- ☐ פונקציות אלה כוללות תמיד __ לפני שם הפונקציה ואחריהם
- ☐ פונקציה חשובה כזו נקראת פונקצית בנאי: __init__()
- ☐ הפונקציה נקראת אוטומטית כשנוצר אובייקט מאותה מחלקה
- ☐ פונקצית בנאי שמשמשת לאתחול משתני אובייקט ספציפי

```
class Person:
    # init method or constructor
    def __init__(self, name):
        self.name = name

    # Sample Method
    def say_hi(self):
        print('Hello, my name is', self.name)

p = Person('Dani')
p.say_hi()
```

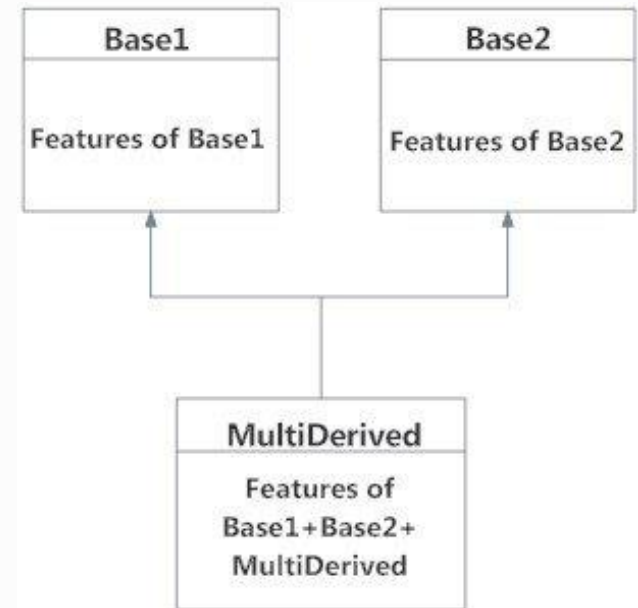
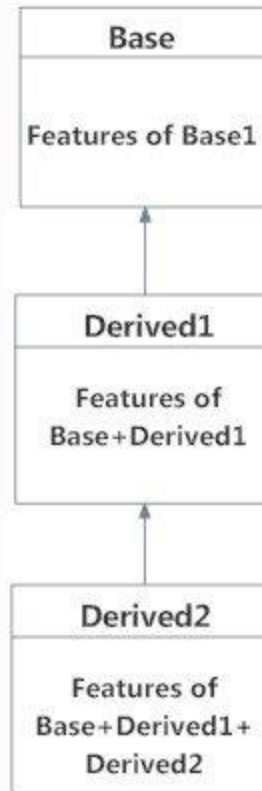


הורשה מרחבה והורשה מדורגת

- Multiple Inheritance – הורשה ממספר מחלקות "בבת אחת" ☐
- Multilevel Inheritance – הורשה ממספר מחלקות בצורה מדורגת ☐

```
class Base1:  
    pass  
  
class Base2:  
    pass  
  
class MultiDerived(Base1, Base2):  
    pass
```

```
class Base:  
    pass  
  
class Derived1(Base):  
    pass  
  
class Derived2(Derived1):  
    pass
```



Reflection

- ☐ האם מחלקה יורשת ממחלקה
- ☐ האם אובייקט הינו מופע מחלקה

```
class Mammal:
    pass

class Dog(Mammal):
    pass

d=Dog()

print(issubclass(Dog, Mammal)) => True !

print(isinstance(d, Dog))  => True !
```



הורשות, שייכות ו- Reflection

- ☐ כל מחלקה בפייתון יורשת ממחלקה אחרת
- ☐ אם לא צוינה הורשה מפורשות, ההורשה היא ממחלקה ששמה *object*
- ☐ הפונקציה `isinstance` מאפשרת לבדוק האם אובייקט שייך למחלקה
- ☐ הפונקציה `issubclass` מאפשרת לבדוק האם מחלקה יורשת ממחלקה

Output: True

```
print(issubclass(list,object))
```

Output: True

```
print(isinstance(5.5,object))
```

Output: True

```
print(isinstance("Hello",object))
```



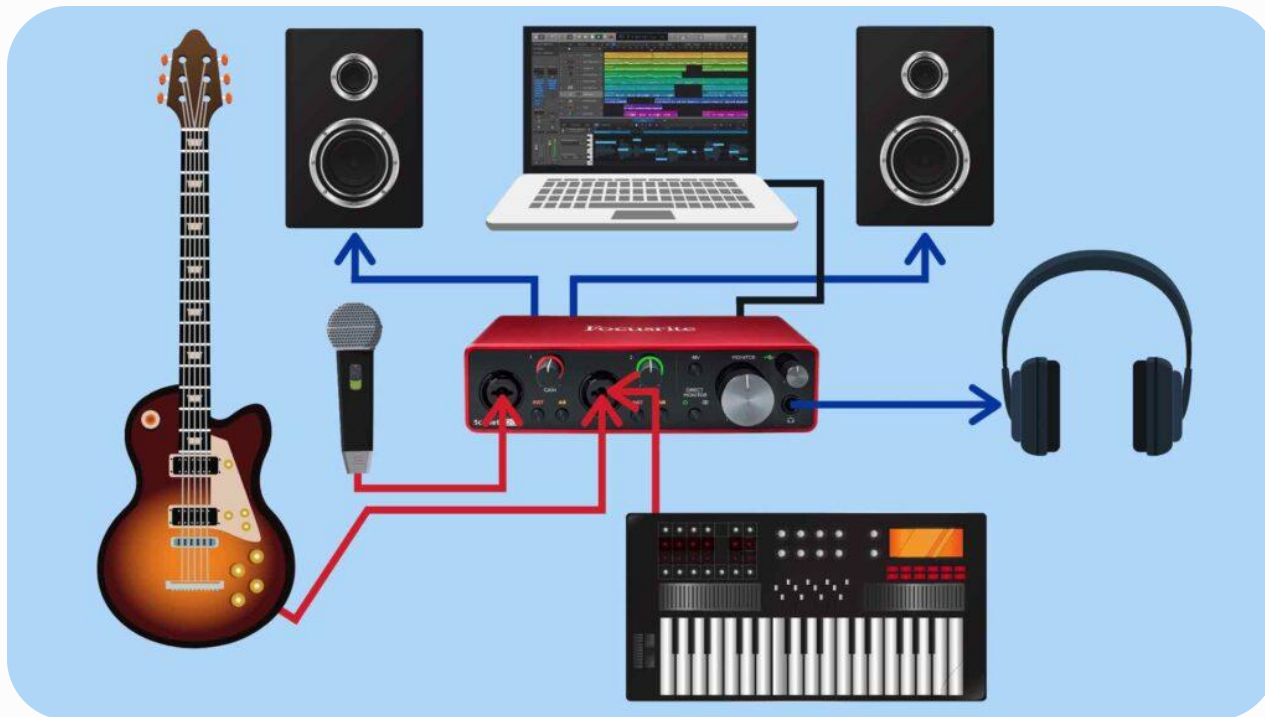
סדר קריאה (MRO)

- ☐ במקרה של הורשה יתן מצב שבאותו Scope יימצאו מספר פונקציות/תכונות עם אותה חתימה
- ☐ הבחירה נעשית בסדר depth-first, left-right
- ☐ לסט הכללים קוראים **Method Resolution Order (MRO)** : מחלקה ספציפית מקבלת קדימות לפני אלו מהם ירש, וכן מסדר ההגדרה במקרה של הורשה מרובה
- ☐ ניתן להשתמש בפונקציה "מיוחדת" בשם `mro()` או בתכונה `__mro__` בכדי לצפות בסדר החיפוש.

```
>>> MultiDerived.__mro__
(<class '__main__.MultiDerived'>,
 <class '__main__.Base1'>,
 <class '__main__.Base2'>,
 <class 'object'>)

>>> MultiDerived.mro()
[<class '__main__.MultiDerived'>,
 <class '__main__.Base1'>,
 <class '__main__.Base2'>,
 <class 'object'>]
```

- ❑ ממשק מייצג **מה** צריך כדי להתחבר ללא פירוט **איך**
- ❑ יתרון: ניתן להחליף מימוש (את מי שמבצע בפועל) מבלי לפגוע בכלל המערכת



<https://realpython.com/python-interface/>

פולימורפיזם מבוסס ממשקים

- הגדרת מחלקה כוללת למעשה שני מרכיבים : ממשק ומימוש
- רב צורתיות הינה היכולת להשתמש באותו ממשק לסוגי משתנים שונים ולקבל התנהגות שונה

```
class Parrot:
    def fly(self):
        print("I am a Flying Parrot")

class Drone:
    def fly(self):
        print("I am a Flying Drone")

# common interface
def flying_test(anyObject):
    anyObject.fly()

#instantiate objects
blu = Parrot()
harop = Drone()

# passing any object
flying_test(blu)
flying_test(harop)
```



מחיקת חברי אובייקט ואובייקטים

- ☐ יצירת אובייקט מקצה זכרון במחשב
- ☐ ניתן גם לשחרר זכרון באמצעות פקודת del

```
p = Person('Dani')  
del p.name  
del p
```



הוספת סוג משתנה להגדרתו

ברירת המחדל בפייתון, בעת הגדרת משתנים, נקראית Dynamic typing ☐

גישה זו יותר גמישה אך עלולה להוביל לשגיאות ☐

פתרון: Static Type Checking ☐

```
def get_first_name(full_name: str)->str:  
    return full_name.split(" ")[0]
```

```
raw_name: str = input("Please enter your name: ")  
first_name: str = get_first_name(raw_name)  
  
print(f"Hi, {first_name}!")
```

```
from typing import Dict, List  
  
name_counts: Dict[str, int] = {  
    "Adam": 10,  
    "Guido": 12  
}  
  
numbers: List[int] = [1, 2, 3, 4, 5, 6]  
  
list_of_dicts: List[Dict[str, int]] = [  
    {"key1": 1},  
    {"key2": 2}  
]
```

<https://realpython.com/python3-object-oriented-programming/>

<https://www.askpython.com/python/examples/easy-games-in-python>



What are Magic Methods

- ❑ Magic methods (also called dunder) in Python are the special methods that start and end with the double underscores
- ❑ Built-in classes in Python define many magic methods (use `dir()` in cli to check)

[Magic or Dunder Methods in Python](#)
[Python Magic Methods](#)

Dunder Functions Demo

- ❑ Code using only one dunder example (`__init__`)

```
class Cup:
    # magic method to initiate object
    def __init__(self, maxVolume, currentVolume):
        self.maxVolume = maxVolume
        self.currentVolume = currentVolume
    def Add(self, delta):
        self.currentVolume += delta

cup1 = Cup(300, 100)
cup2 = Cup(300, 50)
cup1.currentVolume += cup2.currentVolume
print(cup1.currentVolume)
```

Dunder Functions Demo

- ❑ Code making use of dunder function `__repr__`
- ❑ This is an example of **Method Overriding**

```
class Cup:
    # magic method to initiate object
    def __init__(self, maxVolume, currentVolume):
        self.maxVolume = maxVolume
        self.currentVolume = currentVolume
    def Add(self, delta):
        self.currentVolume += delta

    def __repr__(self):
        return f'I am a cup holding {self.currentVolume} cc. water, max is {self.maxVolume}'

cup1 = Cup(300, 100)
cup2 = Cup(300, 50)
cup1.currentVolume += cup2.currentVolume
print(cup1)
```



Dunder Functions Demo

- ❑ Code making use of dunder function `__add__`
- ❑ This is an example of **Operator Overloading**

```
class Cup:
    # magic method to initiate object
    def __init__(self, maxVolume, currentVolume):
        self.maxVolume = maxVolume
        self.currentVolume = currentVolume

    def __add__(self, other):
        total = self.currentVolume + other.currentVolume
        print(total)
        if total > self.maxVolume:
            self.currentVolume = self.maxVolume
        elif total < 0:
            self.currentVolume = 0
        else:
            self.currentVolume = total
        return self

cup1 = Cup(300, 100)
cup2 = Cup(300, 150)

cup1 = cup1 + cup2 # !!!
print(cup1)
```

